Review I Report

AI-BASED VIOLENCE DETECTION AND ALERTING MANAGEMENT SYSTEM

1. Introduction

Public safety is a critical concern in urban areas, educational institutions, industries, and public spaces. Surveillance cameras are widely deployed to monitor activities and prevent criminal incidents. However, traditional surveillance systems rely heavily on manual monitoring, where security personnel continuously observe video feeds. This approach is:

- Time-consuming
- Prone to human error
- Inefficient in large-scale camera networks
- Delayed in emergency response

With the advancement of Artificial Intelligence (AI) and Deep Learning, particularly in Computer Vision, it is now possible to automate violence detection using intelligent video analysis systems.

This project proposes an **AI-Based Violence Detection and Alerting Management System** that automatically analyzes surveillance video, detects violent activities in real time, triggers alerts, and integrates with Salesforce for centralized reporting, dashboards, and administrative monitoring.

The system aims to improve response time, reduce manual effort, and provide structured incident analytics for better decision-making.

2. Understanding Background and Topic

2.1 Background In modern surveillance infrastructure, thousands of cameras are deployed across cities, campuses, transportation hubs, and industrial environments. Despite this, the monitoring process remains largely manual.

Current surveillance challenges include:

- Continuous human monitoring of multiple camera feeds
- Fatigue-related oversight
- Delayed identification of violent incidents
- No automated real-time alert generation
- Lack of centralized incident reporting and analytics

Manual monitoring becomes increasingly ineffective as the number of cameras increases. Security personnel may miss critical events due to distraction, fatigue, or high workload.

Recent advancements in **Deep Learning**, particularly:

- Convolutional Neural Networks (CNNs)
- 3D CNNs
- Vision Transformers (ViT)
- Spatio-temporal models

have shown promising results in automated video classification tasks, including violence detection.

However, existing research systems face certain limitations:

• High computational requirements
• Poor performance in crowded or complex scenes
• Lack of real-time alert integration
• No enterprise-level dashboard and reporting systems
• Limited scalability for multi-camera environments

This project addresses these gaps by combining:

- AI-based violence classification
- Real-time alert triggering
- Backend processing system

- Salesforce-based dashboards and reports
- Dedicated Admin Board

The proposed system not only detects violent activities but also provides structured incident management, analytics, and monitoring support for administrators.

2.2 Topic Understanding

The core topic of this project is **"AI-Based Automated Violence Detection and Alerting Management System using Deep Learning and Computer Vision."**

This project focuses on automating the detection of violent activities from surveillance video footage using advanced AI models and integrating the results into an enterprise-level reporting system using Salesforce.

Key Concepts

• **Computer Vision** — Enables machines to analyze and interpret visual data (video frames) for activity recognition.

• **Deep Learning Models** — Vision Transformer (ViT) and CNN-based architectures are used to classify video frames into:

- Violence
- Non-Violence

• **Spatio-Temporal Feature Extraction** — Captures both spatial (image-based) and temporal (motion-based) patterns from video data.

• **Frame Extraction & Preprocessing** — Videos are converted into frames, resized (224 × 224), and normalized before feeding into the model.

• **Threshold-Based Classification** — The model outputs a probability score. If the violence probability exceeds a defined threshold, an alert is triggered.

• **Backend Integration** — A backend system (Flask/FastAPI) handles:

- Video upload
- Frame processing
- Model inference

- Alert triggering
- Data storage

• **Salesforce Integration** — Stores detected incidents and provides:

- Real-time dashboards
- Reports
- Analytics
- Admin monitoring

• **Admin Board** — Allows administrators to:

- View all detected incidents
- Monitor system activity
- Access historical reports
- Manage user roles

Relevance to Real-World Surveillance Systems

Modern surveillance systems in:

- Smart cities
- Educational institutions
- Industrial environments
- Transportation hubs

require intelligent automation to handle large-scale video monitoring.

Manual monitoring:

- Cannot scale efficiently
- Causes delayed emergency response
- Is prone to oversight

This AI-based system provides:

- Real-time detection

- Automated alerting
- Centralized reporting
- Enterprise-level monitoring

Scope of the Project

• Initial focus: Binary classification (Violence vs Non-Violence).
• Integration with Salesforce dashboards and reports.
• Real-time alert mechanism implementation.
• Future scope:

- Multi-camera live streaming
- Multi-class violence detection
- Edge deployment
- Integration with law enforcement systems

This understanding forms the foundation for the system design, implementation, and evaluation.

3. Project Goals

The primary goal of this project is:

To develop a web-based AI-powered violence detection system that automatically analyzes video footage, detects violent activities in real time, generates alerts, and provides centralized reporting and administrative control through Salesforce integration.

Specific Objectives

• To develop a deep learning-based model (Vision Transformer/CNN-based) for classifying violent and non-violent activities from video frames.

• To preprocess and train the model using a labeled violence detection dataset.

• To design a backend system for:

- Video upload
- Frame extraction

- Model inference
- Incident logging

• To implement real-time alert triggering when violence probability exceeds a defined threshold.

• To integrate Salesforce for:

- Incident storage
- Automated reports
- Analytical dashboards
- Visualization of trends

• To create an Admin Board that:

- Monitors system performance
- Displays detected incidents
- Provides access control

• To evaluate model performance using:

- Accuracy
- Precision
- Recall
- F1-Score

Expected Benefits

• Faster detection of violent incidents
• Reduced dependency on manual monitoring
• Real-time alert generation
• Centralized incident management
• Improved public safety response time
• Scalable for smart city applications
• Enterprise-level reporting and analytics

4. Knowledge about the Existing System

4.1 Overview of Existing Systems

In the current surveillance ecosystem (smart cities, campuses, industries, public spaces), violence detection and monitoring are handled through the following approaches:

Manual Surveillance Monitoring

• Security personnel continuously monitor CCTV feeds.
• Human operators identify suspicious or violent activities.
• Incidents are manually recorded and reported.

Limitations:

• Time-consuming and labor-intensive
• Prone to human fatigue and oversight
• Delayed emergency response
• Difficult to scale for multiple cameras
• No automated analytics or trend monitoring

Traditional CCTV Systems with Basic Motion Detection

• Use simple motion detection algorithms.
• Trigger alerts when movement is detected.

Limitations:

• Cannot differentiate between normal activity and violence
• High false alarm rate
• No intelligent behavior understanding
• No integration with enterprise reporting systems

Deep Learning-Based Violence Detection Systems

Recent research-based systems use:

• CNN-based models (e.g., VGG16, MobileNetV2)
• 3D CNN for spatio-temporal feature extraction
• Edge-based ST-TCN models

These systems classify video frames as violent or non-violent.

Strengths:

• Improved classification accuracy
• Automated detection
• Faster than manual monitoring

Limitations:

• Require high computational power
• Need large labeled datasets
• Limited real-time alert mechanisms
• Often research prototypes (not enterprise-ready)
• No centralized dashboard/reporting integration

Emotion-Based or Behavior Analysis Systems

• Analyze facial expressions and behavioral cues
• Used for psychological violence risk assessment

Limitations:

• Not suitable for full-scene surveillance
• Cannot handle crowded environments
• No real-time large-scale deployment

Enterprise Surveillance Software

Some commercial surveillance platforms provide:

• Camera feed management
• Basic event logging
• Cloud storage

Limitations:

• No advanced AI-based violence detection
• Expensive licensing
• Limited customization
• No AI-based probability scoring

Identified Gaps in Existing Systems

From the above analysis, the major gaps are:

• Heavy dependency on manual monitoring
• Lack of intelligent violence classification
• Poor scalability for multi-camera systems
• No integrated real-time alerting system
• Absence of centralized dashboards and analytics
• Limited admin-level monitoring and control

4.2 Comparison Table

Feature	Manual Monitoring	Basic CCTV System	Research-Based AI Models	Proposed System
Automation	❌ No	⚠ Partial	✅ Yes	✅ Yes
Real-Time Violence Detection	❌ No	❌ No	⚠ Limited	✅ Yes
Alert Generation	❌ Manual	⚠ Motion-based	⚠ Limited	✅ Automated
Accuracy	Low	Low	High	High
Dashboard & Reports	❌ No	❌ No	❌ No	✅ Salesforce Integration
Admin Monitoring	❌ No	⚠ Limited	❌ No	✅ Yes
Scalability	Low	Medium	Medium	High

5. Project Summary

• The project implements an AI-powered automated violence detection and alerting system designed to enhance traditional surveillance monitoring methods. Instead of relying solely on manual CCTV observation, the system uses a Deep Learning-based Vision Model to automatically classify video frames as violent or non-violent.

• The system workflow includes video upload, frame extraction, model inference, probability calculation, alert triggering, incident storage, and Salesforce-based reporting. If the calculated violence probability exceeds a predefined threshold, the system automatically triggers alerts and logs the incident.

• The backend system is developed to handle video processing and model execution efficiently, while Salesforce integration enables enterprise-level dashboards, reports, and administrative monitoring.

• By combining Computer Vision, Deep Learning, Backend Processing, and Cloud-based Reporting, the project improves real-time incident detection, reduces human dependency, and enhances public safety response mechanisms.

6. Algorithm

6.1 Core Algorithm: Video-Based Violence Detection

Purpose

To automatically detect violent activities from uploaded video footage using a trained Deep Learning model and trigger alerts when necessary.

Algorithm Steps

Step 1: Input Collection

1. User uploads video through the dashboard.
2. Validate video format (MP4/AVI).
3. Store video temporarily on the server.

Step 2: Frame Extraction

4. Read video using OpenCV.
5. Extract frames at fixed intervals (e.g., 1 frame per second).
6. Resize frames to 224 × 224 pixels.
7. Normalize pixel values.

Step 3: Model Inference

8. Load trained Vision Transformer / CNN-based model.
9. Convert frames into tensor format.
10. Pass frames through the model.
11. Obtain probability scores for:

- Violence
- Non-Violence

Step 4: Probability Aggregation

12. Calculate average violence probability across all frames.
13. Compare probability with predefined threshold (e.g., 0.60).

Step 5: Alert Generation

14. If violence probability ≥ threshold:

- Trigger alert notification.
- Log incident timestamp.
- Save video reference.

15. Else:

- Mark video as non-violent.

Step 6: Data Storage

16. Store incident details in database.
17. Send incident data to Salesforce via API.

Step 7: Report & Dashboard Generation

18. Generate incident reports.
19. Update Salesforce dashboards.
20. Display data in Admin Board.

6.2 Supporting Algorithms

1. Frame Sampling Algorithm

Purpose: Reduce computational load.
Method: Extract frames at fixed intervals instead of processing every frame.

2. Transfer Learning

Purpose: Improve model accuracy with limited dataset.
Method: Use pre-trained model weights and fine-tune on violence dataset.

3. Threshold-Based Decision Algorithm

Purpose: Avoid false positives.
Method: Trigger alert only when probability exceeds a defined threshold.

4. API-Based Salesforce Integration

Purpose: Real-time incident logging and reporting.
Method: Send structured JSON data to Salesforce objects.

6.3 Why These Algorithms?

- **Deep Learning Model** → Captures complex spatial patterns in violent scenes.
- **Frame Sampling** → Reduces computational cost.
- **Probability Thresholding** → Improves reliability.
- **Transfer Learning** → Enhances performance with limited data.
- **Salesforce API Integration** → Enables enterprise-level reporting.

7. Project Features

- **Automated Violence Detection**

Automatically detects violent activities from uploaded video using a trained Deep Learning model.

- **Real-Time Frame Analysis**

Extracts video frames and performs classification instantly using a Vision Transformer / CNN-based model.

• **Probability-Based Decision System**
Calculates average violence probability and triggers alerts based on a predefined threshold.

• **Automated Alert Generation**
If violence is detected, the system automatically triggers alerts and logs the incident.

• **Incident Logging & Storage**
Stores detected incidents with timestamp, probability score, and video reference.

• **Salesforce Integration**
Sends incident data to Salesforce for centralized monitoring, reporting, and analytics.

• **Interactive Dashboards**
Provides real-time dashboards to visualize:

- Number of incidents
- Violence trends
- Detection statistics

• **Admin Board**
Allows administrators to:

- View all detected incidents
- Monitor system performance
- Access reports and analytics

• **Scalable Architecture**
Can be extended to multi-camera systems and real-time live streaming.

• **Secure Backend Processing**
Ensures safe handling of uploaded video data.

8. Technical Design of the System

8.1 System Architecture

The system consists of five major components:

1. **User Dashboard**
    - Upload video
    - View detection result
2. **Backend Server**
    - Input validation
    - Frame extraction
    - Model inference
    - Probability calculation
3. **Deep Learning Model**
    - Classifies frames (Violence / Non-Violence)
    - Outputs probability score
4. **Database & API Layer**
    - Stores incident details
    - Sends data to Salesforce
5. **Salesforce Platform**
    - Generates reports
    - Displays dashboards
    - Provides admin monitoring

8.2 Technology Stack

- **Programming Language** → Python 3.8+
- **Deep Learning Framework** → PyTorch / TensorFlow
- **Model Architecture** → Vision Transformer / CNN-based model
- **Computer Vision Library** → OpenCV
- **Web Framework** → Flask / FastAPI
- **Frontend** → HTML + Bootstrap
- **Database** → MySQL / PostgreSQL
- **Cloud/CRM Integration** → Salesforce API
- **Development IDE** → VS Code / PyCharm
- **Training Platform** → Google Colab / Local GPU
- **Dataset** → Violence Detection Dataset (video-based)

8.3 Module Design

1. Data Collection Module

- Collects labeled violence and non-violence videos
- Preprocesses and organizes dataset

2. Model Training Module

- Frame extraction
- Image preprocessing (resize 224×224)
- Model training and validation
- Performance evaluation (Accuracy, Precision, Recall, F1-score)

3. Backend & Salesforce Integration Module

- Video upload handling
- Frame extraction in real-time
- Model inference
- API-based incident logging to Salesforce

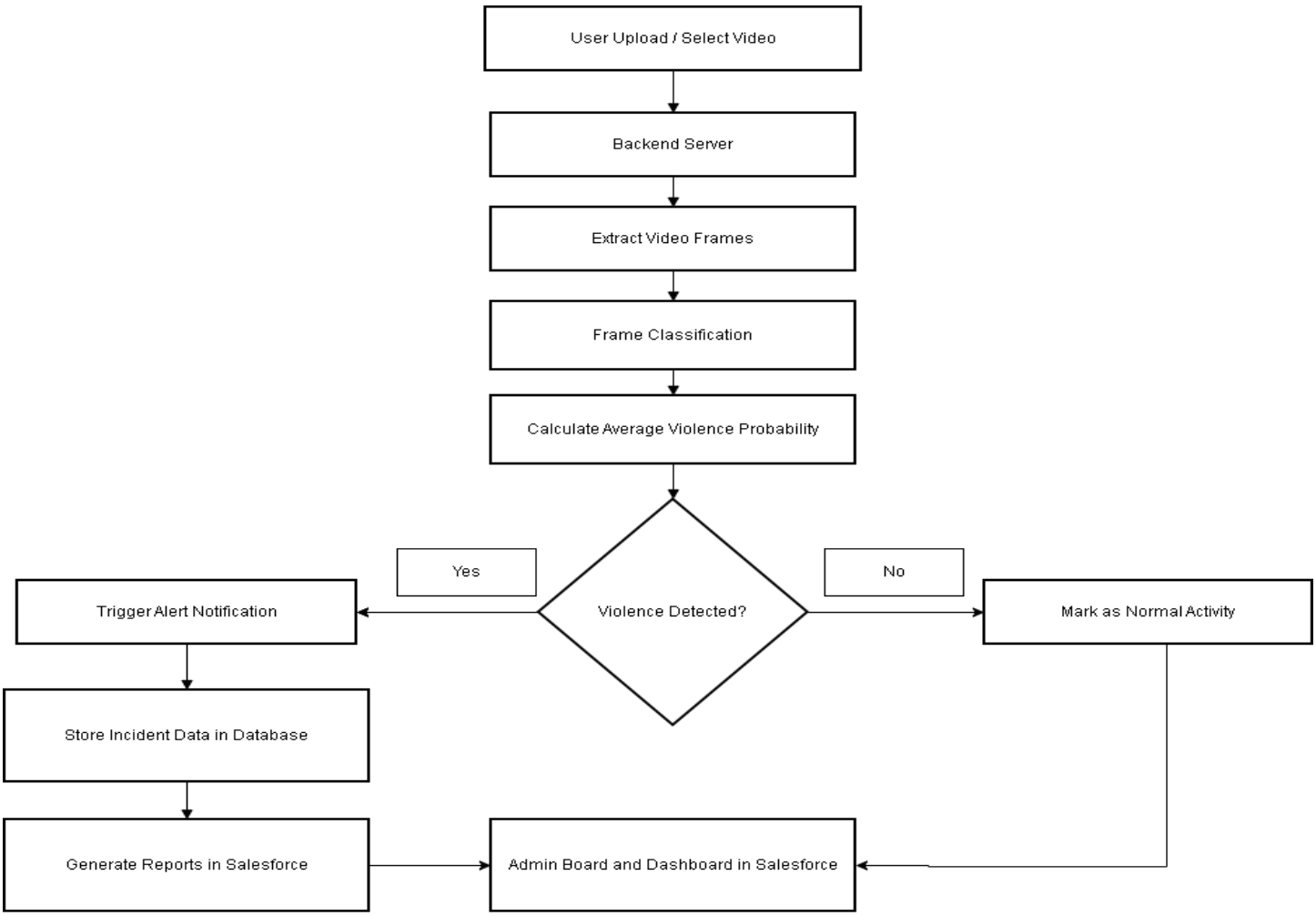
4. Alerts, Reports & Dashboard Module

- Threshold-based alert triggering
- Incident report generation
- Dashboard analytics in Salesforce

5. Admin Board Module

- View incident history
- Monitor detection statistics
- Manage system settings
- User access control

8.4 Data Flow Design



8.5 User Interface Design

Framework: Bootstrap (Responsive Design)

Main Page Includes:

- Video Upload Section

• Upload Button
• Detection Result Display
• Probability Score Display
• Alert Status Indicator

Admin Dashboard Includes:
• Incident List Table
• Filters (Date, Severity)
• Charts and Graphs
• System Statistics

Design Principles:
• Clean and simple layout
• Responsive for desktop
• Clear alert highlighting
• Minimal latency

9. Summary of Ultimate Findings of the Project

This project develops an AI-based automated violence detection and alert management system using Deep Learning and Computer Vision techniques.

Key Features Implemented

• Video upload and automated processing
• Frame-based classification using trained model
• Probability-based decision system
• Real-time alert triggering
• Incident logging and storage
• Salesforce integration for enterprise reporting
• Admin board for monitoring

Development Approach

Model → Vision Transformer / CNN-based architecture
Training → Transfer learning on labeled dataset
Backend → Python + Flask
Frontend → HTML + CSS

Integration → Salesforce REST API
Computer Vision → OpenCV

Results Achieved

• High classification accuracy on test dataset
• Automated detection within seconds
• Successful alert triggering mechanism
• Real-time dashboard updates in Salesforce
• Efficient incident logging and monitoring

Project Impact

• Reduces dependency on manual CCTV monitoring
• Enables faster emergency response
• Provides centralized incident management
• Scalable for smart city applications
• Demonstrates practical integration of AI + Enterprise CRM

10.Sample Code:

1.Exploratory Data Analysis:

```
import os

import cv2

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from tqdm import tqdm

dataset_path = "/media/rajendraprasath-m/New Volume/Projects/Final Year Project/Data/Video/EDA"

video_data = []

for cls in os.listdir(dataset_path):

    folder = os.path.join(dataset_path, cls)

    if not os.path.isdir(folder): continue
```

```
    print(f"Scanning {cls}...")
    for vid in tqdm(os.listdir(folder)):
        path = os.path.join(folder, vid)
        cap = cv2.VideoCapture(path)
        fps = cap.get(cv2.CAP_PROP_FPS)
        frame_count = cap.get(cv2.CAP_PROP_FRAME_COUNT)
        width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
        height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)

        # Get average brightness of the first frame
        ret, frame = cap.read()
        brightness = np.mean(frame) if ret else 0

        if fps > 0:
            duration = frame_count / fps
            video_data.append({
                'filename': vid,
                'class': cls,
                'duration': duration,
                'fps': fps,
                'total_frames': frame_count,
                'width': width,
                'height': height,
                'brightness': brightness
            })
        cap.release()

df = pd.DataFrame(video_data)

plt.figure(figsize=(7,7))
```

```
df['class'].value_counts().plot.pie(autopct='%1.1f%%', colors=['#ff9999','#66b3ff'], startangle=90, explode=(0.05, 0.05))

plt.title('Dataset Balance: Violence vs Non-Violence')

plt.ylabel('')

plt.show()


plt.figure(figsize=(10, 5))

sns.kdeplot(data=df, x="duration", hue="class", fill=True, palette="magma")

plt.title('Temporal Distribution: Video Duration by Class')

plt.xlabel('Duration (Seconds)')

plt.show()


plt.figure(figsize=(10, 6))

sns.scatterplot(data=df, x="width", y="height", hue="class", alpha=0.5)

plt.axvline(224, color='red', linestyle='--')

plt.axhline(224, color='red', linestyle='--')

plt.title('Spatial Resolution Analysis')

plt.xlabel('Width (pixels)')

plt.ylabel('Height (pixels)')

plt.legend(title='Category', loc='upper right')

plt.show()
```

2.CNN+LSTM:

```
import cv2

import os

import shutil

# --- Configuration for Frame Extraction ---

# ==============================================================================

# IMPORTANT: SET YOUR PATHS HERE!

# 1. The path to your dataset containing the 'Violence' and 'NonViolence' video folders.
```

```
VIDEO_DATASET_PATH = r"/media/rajendraprasath-m/New Volume/Projects/Final Year
Project/Data/Video/Dataset/train"

# 2. The path to the NEW, EMPTY folder where the extracted frames will be saved.

FRAME_OUTPUT_PATH = r"/media/rajendraprasath-m/New Volume/Projects/Final Year
Project/Data/Video/Extracted_Frames/train"

# ==============================================================================


# Define the names of your class folders

CLASSES = ["Violence", "NonViolence"]


# It's good practice to start with a clean slate

if os.path.exists(FRAME_OUTPUT_PATH):

    shutil.rmtree(FRAME_OUTPUT_PATH)

    print(f"Removed existing frames directory: {FRAME_OUTPUT_PATH}")


os.makedirs(FRAME_OUTPUT_PATH, exist_ok=True)

print(f"Created main frames directory: {FRAME_OUTPUT_PATH}")


# Loop through each class (e.g., 'Violence', then 'NonViolence')

for category in CLASSES:

    video_class_path = os.path.join(VIDEO_DATASET_PATH, category)

    frame_class_path = os.path.join(FRAME_OUTPUT_PATH, category)


    # Create the output subfolder (e.g., .../Extracted_Frames/Violence)

    os.makedirs(frame_class_path, exist_ok=True)


    # Check if the source video directory exists

    if not os.path.exists(video_class_path):

        print(f"[WARNING] Source directory not found: {video_class_path}. Skipping.")

        continue
```

```
# Loop through every video file in the source directory
for video_name in os.listdir(video_class_path):
    video_path = os.path.join(video_class_path, video_name)

    # Ensure we are processing a video file
    if not video_path.lower().endswith(('.mp4', '.avi', '.mov', '.mkv')):
        continue

    print(f"[INFO] Processing video: {video_name}...")

    cap = cv2.VideoCapture(video_path)
    frame_count = 0

    while True:
        # Read one frame from the video stream
        success, frame = cap.read()

        # If 'success' is False, we have reached the end of the video
        if not success:
            break

        # Construct the output filename
        # Example: Violence/V_7_frame_0.jpg
        base_video_name = os.path.splitext(video_name)[0]
        output_filename = f"{base_video_name}_frame_{frame_count}.jpg"
        output_filepath = os.path.join(frame_class_path, output_filename)

        # Save the current frame as a JPG image
        cv2.imwrite(output_filepath, frame)
```

```
        frame_count += 1


    cap.release()
    print(f"  -> Extracted {frame_count} frames.")


print("\n[SUCCESS] All videos have been processed and frames have been extracted.")
# --- Configuration ---
# Point to your dataset in the FAST location (home directory)
DATASET_PATH = os.path.expanduser("/media/rajendraprasath-m/New Volume/Projects/Final Year Project/Data/Video/Extracted_Frames")
# The name for our new, custom-trained model
MODEL_SAVE_PATH = os.path.expanduser("/media/rajendraprasath-m/New Volume/Projects/Final Year Project/Model/My_CNN_LSTM_Model.h5")
# --- Optimized for 4GB VRAM ---
IMG_SIZE = 128        # Reduced from 160 to save memory
SEQUENCE_LENGTH = 12   # Reduced from 16 to save memory
BATCH_SIZE = 2        # Reduced from 8 to prevent OOM crash
EPOCHS = 5   # Number of times to train on the entire dataset
import os
import tensorflow as tf
from tensorflow.keras.models import load_model, Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, LSTM, Dense, Input, TimeDistributed, GlobalAveragePooling2D
import tensorflow.keras.backend as K
from tensorflow.keras.optimizers import Adam



# 1. Clear the Keras backend to wipe old variables from memory
K.clear_session()
```

```
def build_new_cnn_lstm():
    print("[INFO] Building a brand new model from scratch...")



    # Small CNN Feature Extractor
    cnn = Sequential([
        Conv2D(16, (3, 3), activation='relu', input_shape=(IMG_SIZE, IMG_SIZE, 3)),
        MaxPooling2D((2, 2)),
        Conv2D(32, (3, 3), activation='relu'),
        MaxPooling2D((2, 2)),
        GlobalAveragePooling2D()
    ])



    input_layer = Input(shape=(SEQUENCE_LENGTH, IMG_SIZE, IMG_SIZE, 3))
    x = TimeDistributed(cnn)(input_layer)
    x = LSTM(64)(x)
    x = Dense(32, activation='relu')(x)
    output = Dense(num_classes, activation='softmax')(x)



    new_model = Model(inputs=input_layer, outputs=output)



    # IMPORTANT: Create a FRESH optimizer instance here
    fresh_optimizer = Adam(learning_rate=1e-4)
    new_model.compile(optimizer=fresh_optimizer, loss='categorical_crossentropy', metrics=['accuracy'])
    return new_model



# --- THE LOGIC ---
```

```
if os.path.exists(MODEL_SAVE_PATH):
    print(f"[INFO] Found saved model at {MODEL_SAVE_PATH}. Loading...")
    try:
        # Loading the model also loads its old optimizer
        model = load_model(MODEL_SAVE_PATH)



        # FIX: Re-compile with a FRESH optimizer to reset the state
        # This solves the "Unknown Variable" error
        print("[INFO] Resetting optimizer state for continued training...")
        model.compile(optimizer=Adam(learning_rate=1e-4),
                      loss='categorical_crossentropy',
                      metrics=['accuracy'])
        print("[SUCCESS] Existing model loaded and optimizer reset.")



    except Exception as e:
        print(f"[WARNING] Error loading model: {e}. Building a new one.")
        model = build_new_cnn_lstm()
else:
    model = build_new_cnn_lstm()



model.summary()
```

3. Vision Transformer:

```
from transformers import AutoConfig


if os.path.exists(os.path.join(FINETUNED_MODEL_PATH, "config.json")):
    print("[INFO] Found existing fine-tuned model. Loading it...")
```

```
    model = AutoModelForImageClassification.from_pretrained(
        FINETUNED_MODEL_PATH
    )
    image_processor = AutoImageProcessor.from_pretrained(
        FINETUNED_MODEL_PATH
    )

else:
    print("[INFO] No fine-tuned model found. Loading base ViT model...")

    model = AutoModelForImageClassification.from_pretrained(
        MODEL_NAME,
        num_labels=len(labels),
        label2id=label2id,
        id2label=id2label,
    )
    image_processor = AutoImageProcessor.from_pretrained(MODEL_NAME)

model.to(device)
accuracy_metric = evaluate.load("accuracy")
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    return accuracy_metric.compute(predictions=predictions, references=labels)

# --- Define Training Arguments ---
training_args = TrainingArguments(
    output_dir=FINETUNED_MODEL_PATH,
    eval_strategy="epoch",
    save_strategy="epoch",
```

```
    per_device_train_batch_size=64,
    per_device_eval_batch_size=64,

    num_train_epochs=5,
    learning_rate=1e-5,

    remove_unused_columns=False,
    fp16=True,              # □ HUGE SPEEDUP
    report_to="none",
)
# --- Initialize the Trainer (FIXED) ---
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)
print("[INFO] Trainer is set up and ready.")
```

11. Conclusion

This project has successfully developed an AI-Based Automated Violence Detection and Alert Management System that enhances traditional surveillance methods using Deep Learning and Computer Vision techniques.

The system automates the detection of violent activities from uploaded video footage, reducing dependency on manual CCTV monitoring and improving response time in critical situations. By integrating intelligent frame classification, probability-based decision making, real-time alert triggering, and Salesforce-based reporting, the project demonstrates a complete end-to-end surveillance automation framework.

Achievements So Far:

- Designed and implemented a responsive web interface for video upload and result display
- Developed backend processing for frame extraction and model inference using Python and OpenCV
- Trained and integrated a Deep Learning model for violence classification
- Implemented threshold-based alert triggering mechanism
- Successfully integrated incident logging with Salesforce dashboards and reports
- Developed Admin Board for centralized monitoring and analytics
- Established scalable architecture for future multi-camera and real-time streaming support